Kresilo

A creative workspace where people collect references, shape them with an AI collaborator, and approve every change themselves. These pages show what the product runs on today, why, what it costs us in tradeoffs, what we looked at instead, the licences involved, and a proposed path for measuring and improving it.

What runs today

Layer	Technology	Why
Interface	React 19, TypeScript, Vite, Tailwind, shadcn/Radix	Largest ecosystem and hiring pool; accessible components we own in the repo
Server	Nitro (Node 24), typed API routes	One validated command path shared by the interface and the terminal
Database	Postgres 16 through the Octo workspace	Relational data with constraints and migrations on a platform the owner controls
AI collaborator	Hosted models through the InferHub gateway: a text model for proposals, a vision model for reading references	Proposals are drafts; nothing changes until a person approves. A rule-based fallback keeps Copilot working if a model fails
Design core (direction)	Open Design: canvas, adapters and motion, to be ported into Kresilo	Owner direction, October 22, 2026. Not built; see the Direction section
Motion	HyperFrames renderer with headless Chrome and FFmpeg	Turns designs into deterministic MP4 loops on our server
Access	Guest sessions live; Google sign-in coded, not yet switched on	Private by default; real sign-in waits on provider credentials
Delivery	Automated tests, lint and type checks; required CI before every merge; live behind a Cloudflare tunnel	Changes reach the live site only after checks pass

Today there is no analytics layer. Answering "which model's proposals do people approve?" means hand-written queries on the product database. That gap is what the proposals later in this document address.

Direction: Open Design as the design core

The owner has chosen Open Design (nexu-io/open-design, Apache-2.0) as the base for how people design in Kresilo: its canvas for UI and UX design, its adapters, and its motion tools, ported into Kresilo's Studio. The owner cites a very large community behind it (over 100,000 GitHub stars); we have not verified that figure. Nothing here is built, and we have not yet run Open Design ourselves.

Flow	Who it is for	How Open Design would be used (proposal)
Beginner	People new to design tools	Guided steps; the Copilot proposes from the person's references and they approve. The canvas shows only what the current step needs
Casual	People who edit but do not want every tool	The canvas with a reduced toolset: layers, size, filters, simple motion from templates
Expert	People who want full control	The full canvas, the command console, all adapters and motion options

To verify first: that Open Design runs on our host, which canvas and adapter parts can be reused, how it fits Kresilo's rule that every project change goes through one typed command service and that proposals change nothing before approval, and whether it runs beside Kresilo as a service or is ported into it.

Tradeoffs we are carrying

The web stack was inherited from a builder template rather than chosen after a written comparison, and the data and hosting choices suit a prototype. Honest tradeoffs:

Area	Tradeoff	Reasonable next step
Single-page app	Public maker and story pages are client-rendered, which is weaker for search discovery	Pre-render public pages when discovery matters
Nitro 3 beta	Pinned to a beta: possible breaking changes	Pin and test upgrades; Express, Fastify or Hono are mature fallbacks
Octo over HTTP	8-second query timeout; the credential is not limited to one schema by the provider, so isolation to our demo schema is enforced in server code; images are stored as base64 rows	Scoped database roles and object storage for media as volume grows
Heavy video render	Needs a browser and FFmpeg on the host; slower than page loads	Queue renders and run them off the web server at scale
One laptop host	A single point of failure with no redundancy or monitoring	Move to a small cloud host before real traffic
Two styling systems	Tailwind utilities and hand-written CSS side by side	Consolidate gradually

What we looked at instead

This comparison was prepared for this document; no earlier written decision record exists. Licences were read from each project's repository on October 22, 2026.

Choice	Alternatives (licence)	Why not now
React + Vite	Next.js (MIT), SvelteKit (MIT), Remix (MIT)	No gain that justifies rewriting a working prototype
Nitro server	Express (MIT), Fastify (MIT), Hono (MIT)	Nitro keeps server and interface in one repo; mature fallbacks exist
Octo Postgres	Supabase (Apache-2.0), Neon (Apache-2.0), self-run Postgres; MongoDB and Redis are source-available	Octo already holds our research and graph data; Supabase or Neon would add scoped roles at the cost of a migration
HyperFrames	Remotion (source-available; paid company licence above 3 employees), raw FFmpeg scripts	HyperFrames is Apache-2.0 and shared with Open Design
Canvas	tldraw (paid licence for production), Excalidraw, Fabric.js, Konva (MIT)	Open Design's canvas is the chosen direction; our own layer editor stays until it is replaced
Open Design	Apache-2.0 agent design workspace with templates, adapters, a design canvas and text-to-motion	Chosen as the design core. Integration route and what ports over are still to be verified
Analytics (future)	PostHog (MIT plus separately licensed ee folder), BigQuery and Snowflake (proprietary), Druid and StarRocks (Apache-2.0), TimescaleDB (Apache-2.0 plus Timescale licence)	DuckDB and Parquet cost nothing to start; ClickHouse is the proposed step when scale demands

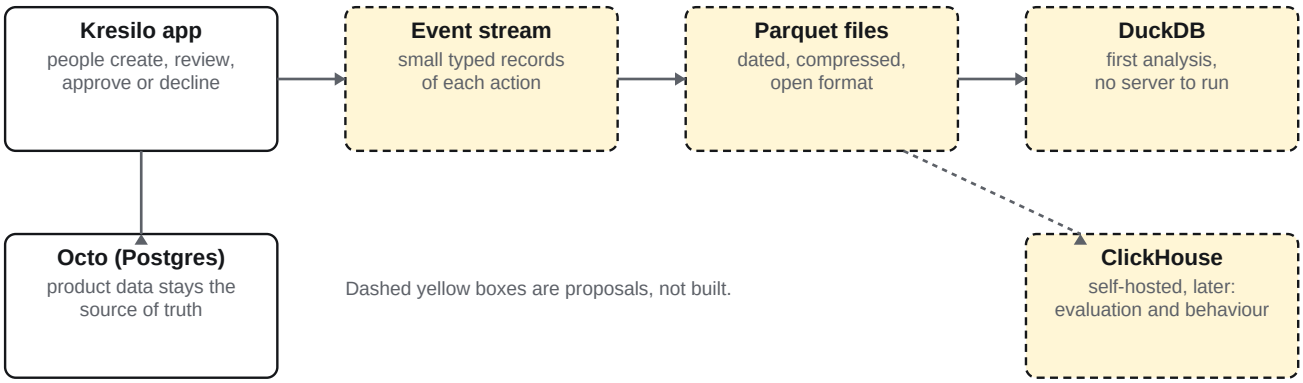
Licence picture

Area	Status
75 direct code packages	Permissive: MIT (most), Apache-2.0, ISC
Animation (GSAP)	GreenSock's own no-charge licence; free for everyone per gsap.com, but not an open-source licence. Terms to be read in full before diligence
Video rendering	HyperFrames is Apache-2.0. The bundled FFmpeg and ffprobe binaries are GPL-licensed builds (their wrappers LGPL-2.1). We run them on our own server and do not ship them to users, the usual basis for using GPL tools in a hosted service; this must be re-checked before any desktop or installer distribution
Proposed analytics	DuckDB is MIT; Parquet and ClickHouse are Apache-2.0. ClickHouse Cloud, if ever used, is a separate commercial service
Avoided	Source-available video and canvas tools that need paid company licences (Remotion, tldraw); AGPL dashboards (Grafana, Plausible, Metabase) unless their network-service obligations are accepted
Hosted AI models	Used as services through the InferHub gateway. The models' own weights licences and the gateway and Octo terms have not been assessed here

A full per-package scan covered 1,885 installed packages; the only copyleft entries were the FFmpeg binaries and one image-library build (Apache-2.0 AND LGPL-3.0-or-later).

Proposed: measuring what works

Every approval or decline is a person's judgement of whether the AI helped. The proposal is to capture those judgements, with their context, as a separate analytics stream so the product database stays fast and the learning does not depend on it.



Proposed layer	Role	Why
Event stream	Typed records: reference kept, proposal approved or declined, edit, error, render	Cheap to add; separates analysis from product data
Parquet files	Durable, compressed, open archive	Any later engine reads the same files; little lock-in
DuckDB (first)	Embedded analytics reading Parquet directly	No new server to run or pay for
ClickHouse, self-hosted (later)	Human-based AI evaluation, behaviour monitoring, operational data, improvement loop	Built for many concurrent queries over large event tables; data stays on infrastructure we control. Cost: we must operate it, on a machine other than today's host

Goal	Example question
Human-based AI evaluation	Which model and prompt produce proposals people approve, once reviewers also rate relevance, grounding in the person's own references, and taste?
Behaviour monitoring	Where do people stop between creating a project and approving a first change? Does each help level help?
Operational data	Did the last release slow the model gateway or the renderer? Which errors do people see?
Improvement loop	Review the weakest proposal types weekly, change the prompt or model, measure the difference

Proposed phases and principles

Phase	What	Exit signal
1	Define the event schema; record events to local files; no new infrastructure	Create, propose, approve, decline, edit and error events flow reliably
2	Close files to Parquet; query with DuckDB; first model-comparison report	One repeatable report the team uses
3	Capture reviewer ratings against a simple rubric	Enough rated proposals to tell two models apart
4	Stand up self-hosted ClickHouse, load history from Parquet, build dashboards	Dashboards used weekly; DuckDB on files has become the bottleneck

Principle	Meaning
People approve	AI proposals never change a design on their own
One source of truth	Product data stays in Postgres; analytics never writes back to it
Secrets stay on the server	No analytics or model keys reach the browser
Open formats	Parquet keeps the data portable between engines
Staged, not speculative	Heavy infrastructure arrives only when volume and questions justify it

Still open

Retention of raw events and pseudonymising of person identifiers; who rates proposals and how disagreements are settled; where ClickHouse runs and who operates it; volume thresholds for moving between phases; model weights licences and platform terms. None of the performance or cost expectations behind this proposal have been measured.

Status: sections marked Proposed are plans under consideration, not commitments. Built items describe the live prototype.